Lecture 6: Regularization: Ridge and Lasso Regression

AI 501: Mathematics for Artificial Intelligence

Fall 2026

Abstract

Lecture 5 ended with a warning. A flexible model fitted by least squares can reach zero training error while predicting nonsense, its coefficients grow enormous, and when features are redundant the matrix $X^\top X$ cannot even be inverted. This lecture repairs all three problems with one idea: add to the loss a price on large weights. Charging the squared length of w gives *ridge regression*, which keeps a closed form, always has a unique solution, and shrinks every weight smoothly. Charging the absolute values gives the *lasso*, which has no closed form but sets some weights exactly to zero and so chooses features for us. We derive both, compute them by hand on the same four fitness-tracker customers, look at the geometry that explains why one shrinks and the other selects, and finish with how to choose the strength of the penalty.

Contents

1 The problem **2**
- 1.1 Three symptoms of one disease . . . 2
- 1.2 The idea: put a price on large weights . . . 3

2 Ridge regression **3**
- 2.1 The ridge loss . . . 3
- 2.2 Leave the intercept alone . . . 3
- 2.3 The closed form . . . 4
- 2.4 Worked example: the age data by hand . . . 4
- 2.5 The inverse always exists . . . 6
- 2.6 Which weights ridge shrinks most . . . 6

3 Units matter: standardize before you penalize **7**

4 Ridge on our problems **7**
- 4.1 Age and sleep: correlated features . . . 8
- 4.2 Taming the cubic . . . 8
- 4.3 The sine wave revisited . . . 9

5 Lasso regression **10**
- 5.1 The lasso loss . . . 10
- 5.2 Why absolute values give exact zeros . . . 11
- 5.3 One weight by hand: soft thresholding . . . 11
- 5.4 Worked example: the age slope . . . 12
- 5.5 Many weights: coordinate descent . . . 13
- 5.6 Age and sleep: the lasso chooses . . . 13
- 5.7 Feature selection in a bigger example . . . 14

6 Ridge or lasso? 15

7 Choosing λ 15
7.1 Never on training error . . . 15
7.2 Leave-one-out on our customers . . . 15
7.3 k-fold cross-validation in practice . . . 16

8 Regularization in practice 16
8.1 Ridge as least squares with extra rows . . . 16
8.2 Cost . . . 17
8.3 The battery problem revisited . . . 17
8.4 Where this leads . . . 18

9 Summary map 18

1 The problem

We are a fitness-tracker company predicting weekly exercise hours y from what we know about a customer. In Lecture 5 we fitted models to four customers by least squares, $w^\star = (X^\mathsf{T}X)^{-1}X^\mathsf{T}y$, which we call *ordinary least squares* (OLS):

customer	age x_1 (years)	sleep x_2 (hours/night)	exercise y (hours/week)
1	25	7	4
2	30	6	5
3	40	6	3
4	50	5	2

With age alone, OLS gave the line $\hat{y} = 7.186 - 0.1017x$ (total squared error 1.186). Adding flexibility exposed three problems.

1.1 Three symptoms of one disease

1. **The cubic memorized the noise.** With four customers and a degree-3 polynomial, least squares passed through every point (training error zero) and then predicted 9.6 hours a week for a 60-year-old. Training error was useless for spotting this.

2. **The coefficients exploded.** In the sine-wave experiment the degree-9 fit had coefficients in the hundreds of thousands, positive and negative, cancelling each other between the data points. Overfit models have huge weights.

3. **Redundant features broke the formula.** When one column of X is (nearly) a combination of others, $X^\mathsf{T}X$ is singular (or nearly so), the inverse does not exist (or is numerically garbage), and individual weights become unreliable. Our age and sleep features had correlation -0.92, and the fitted sleep weight came out with an implausible sign.

These are three faces of one disease: the model has more freedom than the data can pin down, and least squares spends that freedom on fitting noise. Lecture 5 listed three remedies: judge on held-out data, collect more data, and *regularize*. This lecture is the third.

1.2 The idea: put a price on large weights

Symptom 2 suggests the cure. If overfit models have huge weights, then make huge weights expensive. Instead of minimizing the data-fit term alone, minimize

$$\underbrace{\tfrac{1}{2}\|y - Xw\|_2^2}_{\text{fit the data}} + \underbrace{\lambda \cdot (\text{size of } w)}_{\text{keep the weights small}} .$$

The number $\lambda \geq 0$ sets the exchange rate between the two goals.

- $\lambda = 0$: no price on size. We get OLS back.
- $\lambda \to \infty$: size is all that matters. Every slope is driven to zero, and the model predicts a constant.
- In between: the model fits the data, but only as well as it can afford to.

Like the polynomial degree M, λ is a hyper-parameter: we choose it, the data do not (Section 7 says how). What remains is to decide what "size of w" means. Two answers dominate practice: the squared length $\frac{1}{2}\|w\|_2^2$ gives *ridge* regression (Section 2), and the ℓ_1 length $\|w\|_1$ gives the *lasso* (Section 5).

2 Ridge regression

We start with the squared-length penalty because it changes least squares the least: the loss is still a smooth bowl, so we can still set the gradient to zero and get a formula. This section derives that formula, checks it by hand on the age line, and then shows what it buys us: a solution that always exists, and shrinkage that is strongest exactly where the data are weakest.

2.1 The ridge loss

For a model with intercept w_0 and slopes $w_{1:d} = (w_1, \ldots, w_d)^\mathsf{T}$, the ridge loss is

$$\boxed{L_\lambda(w) = \tfrac{1}{2}\|y - Xw\|_2^2 + \tfrac{\lambda}{2}\,\|w_{1:d}\|_2^2 = \tfrac{1}{2}\sum_{i=1}^{n} e_i^2 + \tfrac{\lambda}{2}\sum_{j=1}^{d} w_j^2.}$$

The $\frac{1}{2}$ in front of λ is the same convenience as before: it cancels the 2 that differentiation produces. In statistics this method is called *ridge regression*; in neural networks the same penalty is called *weight decay* or *ℓ_2 regularization*.

2.2 Leave the intercept alone

The penalty sums from $j = 1$: the intercept is not penalized. Penalizing slopes expresses a preference for a flat relationship; penalizing w_0 would instead pull every prediction towards 0 hours, a point fixed by the units, not the data. In matrix form we write the penalty as $\frac{\lambda}{2} w^\mathsf{T} D w$ with $D = \mathrm{diag}(0, 1, \ldots, 1)$. Equivalently, centre every feature and y; then the intercept decouples as $w_0 = \bar{y} - \bar{x}^\mathsf{T} w_{1:d}$ and the slopes solve the same problem with $D = I$ and no column of ones. (Beware: the textbook formula $(X^\mathsf{T}X + \lambda I)^{-1}X^\mathsf{T}y$ applied to an X that still contains the column of ones *does* penalize the intercept.)

2.3 The closed form

Expand the loss as for ordinary least squares:

$$L_\lambda(w) = \tfrac{1}{2}\big(y^\mathsf{T}y - 2w^\mathsf{T}X^\mathsf{T}y + w^\mathsf{T}X^\mathsf{T}Xw\big) + \tfrac{\lambda}{2}\, w^\mathsf{T}Dw.$$

The only new term is another quadratic form, $w^\mathsf{T}(\lambda D)w$ with λD symmetric, so its gradient is λDw (the rule $\nabla(w^\mathsf{T}Pw) = 2Pw$ for symmetric P, with the $\frac{1}{2}$ cancelling the 2). Hence

$$\nabla L_\lambda(w) = X^\mathsf{T}Xw - X^\mathsf{T}y + \lambda Dw.$$

Setting it to zero gives the *regularized normal equations* and their solution:

$$\boxed{(X^\mathsf{T}X + \lambda D)\, w = X^\mathsf{T}y \quad\Longrightarrow\quad w^\star_\lambda = (X^\mathsf{T}X + \lambda D)^{-1}X^\mathsf{T}y.}$$

With centred data and no column of ones, D is the identity and this is the form usually quoted:

$$w^\star_\lambda = (X^\mathsf{T}X + \lambda I)^{-1}X^\mathsf{T}y.$$

Compare with OLS: the only change is $+\lambda$ added to the diagonal of $X^\mathsf{T}X$ (the "ridge" along the diagonal that gives the method its name). Setting $\lambda = 0$ recovers the OLS normal equations exactly.

2.4 Worked example: the age data by hand

We redo the worked example of Lecture 5, the straight line $\hat{y} = w_0 + w_1 x$ for the four customers, now with $\lambda = 100$. The data are unchanged: X has rows $(1, 25), (1, 30), (1, 40), (1, 50)$ and $y = (4, 5, 3, 2)^\mathsf{T}$, so

$$X^\mathsf{T}X = \begin{bmatrix} 4 & 145 \\ 145 & 5625 \end{bmatrix}, \qquad X^\mathsf{T}y = \begin{bmatrix} 14 \\ 470 \end{bmatrix}, \qquad \lambda D = \begin{bmatrix} 0 & 0 \\ 0 & 100 \end{bmatrix}.$$

The penalty touches only the slope, so only the bottom-right entry of the matrix changes. The table follows Lecture 5's calculation step by step (left) next to the ridge version (right).

	OLS (Lecture 5)	ridge, $\lambda = 100$
matrix to invert	$\begin{bmatrix} 4 & 145 \\ 145 & 5625 \end{bmatrix}$	$\begin{bmatrix} 4 & 145 \\ 145 & \mathbf{5725} \end{bmatrix}$
determinant	$22500 - 21025 = 1475$	$22900 - 21025 = 1875$
numerator, entry 1	$5625 \cdot 14 - 145 \cdot 470 = 10600$	$5725 \cdot 14 - 145 \cdot 470 = 12000$
numerator, entry 2	$-145 \cdot 14 + 4 \cdot 470 = -150$	$-145 \cdot 14 + 4 \cdot 470 = -150$
$w^\star = (w_0, w_1)$	$(7.186,\ -0.1017)$	$(6.4,\ -0.08)$
predictions	$4.644,\ 4.136,\ 3.119,\ 2.102$	$4.4,\ 4.0,\ 3.2,\ 2.4$
residuals	$(-38,\ 51,\ -7,\ -6)/59$	$-0.4,\ 1.0,\ -0.2,\ -0.4$
total squared error	$4130/3481 \approx 1.186$	$0.16 + 1 + 0.04 + 0.16 = 1.36$

(Each $w^\star$ is the numerator divided by the determinant. The numerator is the matrix with its diagonal entries swapped and its off-diagonal entries negated, times $X^\mathsf{T}y$.) The second entry of the numerator is the same in both columns, but it is now divided by a larger determinant, so the slope shrinks from -0.1017 to -0.08, about 20% flatter. The ridge line is $\hat{y} = 6.4 - 0.08x$.

Three checks are worth doing.

- **The residuals still sum to zero** ($-0.4 + 1 - 0.2 - 0.4 = 0$), and the line still passes through the point of averages ($6.4 - 0.08 \cdot 36.25 = 3.5$). Both follow from the first regularized normal equation, which is unchanged because the intercept is not penalized.

- **Training error went up**, from 1.186 (OLS) to 1.36. It must: OLS is by definition the line with the smallest squared error. Ridge deliberately buys smaller weights with a little training error.
- **The ridge objective went down.** For OLS, $\frac{1}{2}(1.186) + \frac{100}{2}(0.1017)^2 = 0.593 + 0.517 = 1.110$. For ridge, $\frac{1}{2}(1.36) + 50(0.08)^2 = 0.68 + 0.32 = 1.00$. Ridge wins on the quantity it minimizes.

The centred shortcut. For one feature the centred form gives a formula you can evaluate in your head. With the means $\bar{x} = 36.25$, $\bar{y} = 3.5$ and the sums $S_{xx} = \sum_i (x_i - \bar{x})^2 = 368.75$ and $S_{xy} = \sum_i (x_i - \bar{x})(y_i - \bar{y}) = -37.5$,

$$w_1(\lambda) = \frac{S_{xy}}{S_{xx} + \lambda}, \qquad w_0(\lambda) = \bar{y} - w_1(\lambda)\,\bar{x}.$$

At $\lambda = 0$ this is the familiar OLS slope S_{xy}/S_{xx}; at $\lambda = 100$ it gives $-37.5/468.75 = -0.08$, matching the matrix calculation. The table and Figure 1 show the whole family.

λ	slope w_1	intercept w_0	total squared error	prediction at age 60
0 (OLS)	-0.1017	7.186	1.186	1.08
100	-0.0800	6.400	1.360	1.60
368.75	-0.0508	5.343	2.140	2.29
1000	-0.0274	4.493	3.222	2.85
∞	0	3.500	5.000	3.50

At $\lambda = S_{xx} = 368.75$ the slope is exactly halved. As $\lambda \to \infty$ the line flattens into the best constant model $\hat{y} = \bar{y} = 3.5$. Ridge therefore slides continuously between the line and the constant, and λ is the slider.

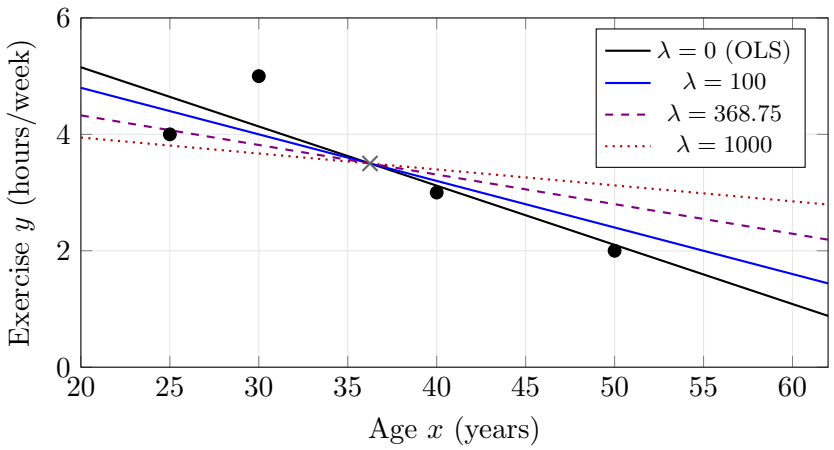


Figure 1: Ridge lines for the four customers. All of them pivot about the point of averages (36.25, 3.5) (the cross), because the intercept is not penalized; increasing λ only flattens the slope.

Check yourself. What happens to $w_\lambda^\star$ as $\lambda \to 0$ and as $\lambda \to \infty$, for a general X?

Answer. As $\lambda \to 0$ it approaches the OLS solution (when that exists). As $\lambda \to \infty$ the slopes behave like $X^\mathsf{T} y/\lambda \to 0$ and the model becomes the constant $\bar{y}$. Ridge never sets a slope exactly to zero for finite λ (unless it was already zero); it only makes it small.

2.5 The inverse always exists

Symptom 3 raised a practical worry: OLS needs $X^\top X$ to be invertible, which requires linearly independent columns, and that fails with too few samples or redundant features. Ridge removes this failure. Work with centred data, so the matrix is $X^\top X + \lambda I$. For any $v \neq 0$,

$$v^\top (X^\top X + \lambda I) v = \|Xv\|_2^2 + \lambda \|v\|_2^2 \;\geq\; \lambda \|v\|_2^2 \;>\; 0 \qquad (\lambda > 0).$$

So $(X^\top X + \lambda I)v$ can never be the zero vector, the matrix is invertible, and the minimizer is unique, *whatever* the data. This holds even with more features than samples ($d > n$) and even with exact duplicate columns. The same inequality is a strict version of the bowl property of least squares, $v^\top X^\top X v = \|Xv\|_2^2 \geq 0$: the loss now curves upwards in *every* direction, so there are no flat valleys of equally good solutions any more.

Check yourself. A 1000×50 feature matrix has feature 50 equal to $2\times$ feature 3, so OLS has no unique solution. Does the ridge solution exist? If instead feature 50 is an *exact copy* of feature 3, how does ridge split the weight between them?

Answer. It exists for every $\lambda > 0$: the argument above never used independent columns. For an exact copy, swapping w_3 and w_{50} leaves both the fit and the penalty unchanged, so if $(w_3, w_{50}) = (a, b)$ is optimal so is (b, a). The minimizer is unique, so $a = b$: ridge splits the weight equally between identical features. (OLS had a whole line of solutions here.)

2.6 Which weights ridge shrinks most

The age example shrank the slope by about 20%. Is that the same for every feature? It is not, and the difference is what makes ridge useful rather than an arbitrary pull towards zero. The one-feature formula of Section 2.4 shows why. Rewrite it as

$$w_1(\lambda) = \frac{S_{xy}}{S_{xx} + \lambda} = \underbrace{\frac{S_{xx}}{S_{xx} + \lambda}}_{\text{shrink factor}} \cdot \underbrace{\frac{S_{xy}}{S_{xx}}}_{\text{OLS slope}} .$$

Ridge multiplies the OLS slope by a number between 0 and 1, and that number depends on $S_{xx} = \sum_i (x_i - \bar{x})^2$, which measures how much the feature *varies* in the data.

- If the feature varies a lot, S_{xx} is large, the factor is close to 1, and ridge barely touches the slope. For our ages, $S_{xx} = 368.75$ and $\lambda = 100$ give a factor of $368.75/468.75 = 0.79$, which is exactly how -0.1017 became -0.080.
- If the feature hardly varies, S_{xx} is small and the factor is close to 0. Had our four customers been aged 35, 36, 37 and 38, then $S_{xx} = 5$ and the same $\lambda = 100$ would give a factor of $5/105 \approx 0.05$.

This is the right behaviour. A slope estimated from customers who are all about the same age is mostly noise: dividing by a small S_{xx} is exactly how OLS turns small fluctuations in y into large, unstable slopes. Ridge shrinks hardest where the data carry the least information.

With several features the same principle applies to *combinations* of features: ridge shrinks hardest along combinations that the data barely vary in, such as "older *and* more sleep" in our customers. Stating this precisely needs eigenvalues, which we meet later in the course; the one-feature case already carries the idea. We see it for age and sleep in Section 4.1.

3 Units matter: standardize before you penalize

Before applying ridge to more than one feature we must settle a practical question: does the answer depend on the units in which the features are recorded? For OLS it does not:

> From Lecture 5
>
> *Check yourself.* Suppose we record age in months instead of years, so every entry in the age column is multiplied by 12. What happens to w_1 and to the predictions?
> *Answer.* w_1 is divided by 12 (a "per month" slope is a twelfth of a "per year" slope), w_0 is unchanged, and the predictions are identical. The same line is described in different units.

With a penalty this is no longer true.

Take $\lambda = 100$ again. In years we found slope -0.08 per year. In months the centred data have $S'_{xx} = 144 \times 368.75 = 53100$ and $S'_{xy} = 12 \times (-37.5) = -450$, so

$$w'_1 = \frac{-450}{53100 + 100} = -0.008459 \text{ per month } = \ -0.1015 \text{ per year.}$$

In months, the same λ barely shrinks the slope at all, and the fitted line is almost the OLS line. The reason is that the penalty λw_1^2 looks at the *number* w_1, and recording age in months makes that number twelve times smaller, so its penalty is 144 times cheaper.

The consequence is a rule: **penalized regression is not unit-free, so put all features on a common scale before fitting**. The standard choice is to standardize each feature, $z_j = (x_j - \bar{x}_j)/s_j$. Then

- a weight means "change in prediction per one standard deviation of the feature", which is comparable across features, so penalizing them equally is fair;
- writing Z for the matrix of standardized features, every diagonal entry of $Z^\mathsf{T} Z$ equals n, so λ has a natural scale ("small" means small compared with n);
- large raw values (such as powers of age, which exceed 100 000) no longer make the computation numerically unreliable.

Only the features are standardized; the intercept is still left unpenalized. Predictions for a new customer use the means and standard deviations computed on the *training* data.

Check yourself. Two features: income in rupees (values around 10^5) and number of children (values 0 to 4). Without standardizing, which weight does ridge effectively penalize more?

Answer. The weight on number of children. Income is a large number, so its weight is tiny and its penalty λw^2 is almost nothing; the children weight is of ordinary size and pays the full price. Ridge would shrink the children effect and leave income almost untouched, for no reason other than units.

4 Ridge on our problems

We now test ridge on the three symptoms of Section 1, one per subsection: redundant features (age and sleep), the cubic that memorized four customers, and the exploding degree-9 polynomial. In each case the features are standardized first, as Section 3 showed we must.

4.1 Age and sleep: correlated features

Symptom 3 was the two-feature model, whose OLS fit gave sleep an implausible negative weight. We standardize both features, using the population standard deviations 9.601 (age) and 0.7071 (sleep), and centre y at $\bar{y} = 3.5$:

$$Z = \begin{bmatrix} -1.172 & 1.414 \\ -0.651 & 0 \\ 0.391 & 0 \\ 1.432 & -1.414 \end{bmatrix}, \quad y_c = \begin{bmatrix} 0.5 \\ 1.5 \\ -0.5 \\ -1.5 \end{bmatrix}, \quad Z^\mathsf{T} Z = \begin{bmatrix} 4 & -3.682 \\ -3.682 & 4 \end{bmatrix}, \quad Z^\mathsf{T} y_c = \begin{bmatrix} -3.906 \\ 2.828 \end{bmatrix}.$$

The diagonal entries equal $n = 4$ (a standardized column has squared length n), and the off-diagonal $-3.682 = 4 \times (-0.92)$ is n times the age–sleep correlation.

The ridge path. Solving $(Z^\mathsf{T} Z + \lambda I)w = Z^\mathsf{T} y_c$ for a range of λ gives the curves in Figure 6 (left), called the *ridge path*. Some values (weights on standardized features, so each means "hours per week per one standard deviation of the feature"):

λ	age weight	sleep weight	total squared error
0 (OLS)	-2.134	-1.257	0.22
0.1	-1.722	-0.857	0.33
0.5	-1.070	-0.247	0.92
1	-0.797	-0.021	1.31
2	-0.580	$+0.115$	1.71

The suspicious negative sleep weight collapses quickly, and by $\lambda \approx 1.1$ it has changed sign. This is the principle of Section 2.6 at work. Our customers who are older almost always sleep less, so the data pin down the *combined* effect of being older and sleeping less, but say almost nothing about how to split it between the two features. OLS splits it in whatever way the noise happens to favour; ridge, which shrinks hardest where the data are least informative, pulls that split back. For large λ the weights become proportional to $Z^\mathsf{T} y_c$, that is, to each feature's own correlation with exercise, and sleep on its own is *positively* related to exercise in our data. Ridge does not tell us which sign is "true" (four customers cannot), but it does stop the model from making confident claims the data do not support.

4.2 Taming the cubic

Symptom 1 was the cubic that passed through all four customers and predicted 9.6 hours at age 60. Powers of raw ages run past 100 000 and make the arithmetic unreliable, so we use the standardized age $z = (x - 36.25)/9.60$ and features z, z^2, z^3, with an unpenalized intercept. (Ridge also lets us fit a cubic to fewer than four points, which we need for validation in Section 7.)

	coefficients on $1, z, z^2, z^3$				predicted hours at age			
λ	w_0	w_1	w_2	w_3	20	35	45	60
0	4.04	-2.50	-0.85	1.12	0.40	4.35	1.90	9.60
0.3	3.89	-1.15	-0.46	0.24	3.34	4.03	2.65	1.95
3	3.73	-0.33	-0.17	-0.23	4.90	3.77	3.12	-1.51

At $\lambda = 0$ this is the cubic of Symptom 1 (same predictions; the coefficients differ only because they multiply powers of z instead of powers of x). A small penalty, $\lambda = 0.3$, cuts the slope coefficients' length from 2.87 to 1.26, gives up a little training error (total squared error 0.59

instead of 0), and brings every prediction back to plausible values. This is the behaviour we wanted.

The last row is a warning. With $\lambda = 3$ the cubic's prediction at age 60 is *negative*. Ridge shrinks the coefficients, but a cubic still bends without limit outside the data, and age 60 is ten years beyond the oldest customer. **Regularization makes a model calmer inside the range of the data; it does not make extrapolation safe.**

4.3 The sine wave revisited

Symptom 2 gives the cleanest demonstration. We repeat the experiment: ten noisy samples of $\sin(2\pi x)$ on $[0, 1]$ (noise standard deviation 0.3), the degree-9 polynomial $w_0 + w_1 x + \cdots + w_9 x^9$, and now a ridge penalty on $w_1, \ldots, w_9$. Following Bishop (Section 1.1), we report λ through $\ln \lambda$ because useful values span many orders of magnitude. To judge each fit on new data, we also draw 200 fresh points from the same process as a validation set.

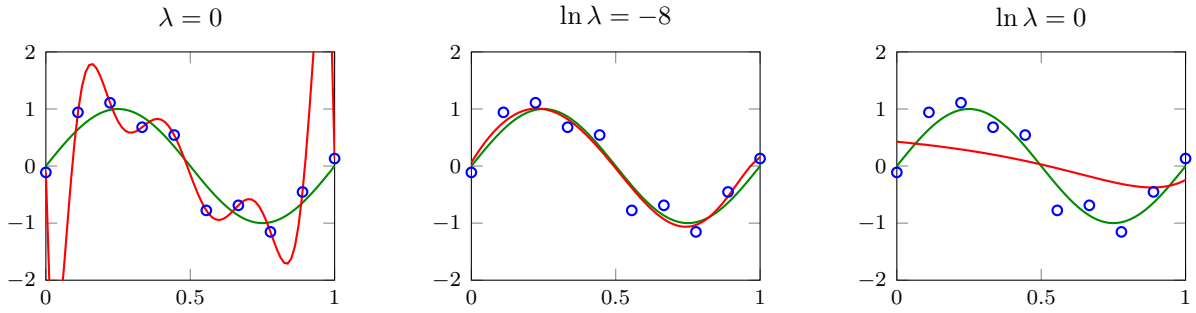


Figure 2: Degree-9 fits to the same ten noisy samples of $\sin(2\pi x)$ (green: true function; blue circles: data; red: fit; simulated, noise standard deviation 0.3). Left: no penalty, the curve hits every point and swings far outside the plot. Middle: a small penalty keeps all nine powers but produces a smooth curve close to the sine. Right: too large a penalty flattens the curve into an underfit.

Figure 2 shows three settings, and the coefficients tell the same story as the curves. (They come from our own simulated draw, so at $\lambda = 0$ they differ from the coefficients in Lecture 5, but they explode in the same way.)

	w_1	w_2	w_3	w_4	w_5	$\cdots$	w_9
$\lambda = 0$	−221.5	5477.6	−50491.8	240732.5	−663880.0	$\cdots$	−128245.8
$\ln \lambda = -18$	15.07	−63.48	117.59	−136.15	53.80	$\cdots$	−13.34
$\ln \lambda = -8$	8.35	−17.98	−3.54	5.31	7.58	$\cdots$	−6.30
$\ln \lambda = 0$	−0.49	−0.46	−0.29	−0.13	−0.00	$\cdots$	0.24

Even a tiny penalty ($\lambda = e^{-18} \approx 1.5 \times 10^{-8}$) cuts the coefficients by a factor of thousands. The reason is the principle of Section 2.6: the powers $x, x^2, \ldots, x^9$ on $[0, 1]$ look very much alike, so there are combinations of them that the data barely vary in, and OLS can only fit the noise by giving such combinations huge, cancelling weights. Because the data barely constrain those combinations, even a tiny penalty outweighs the little they add to the fit, and the huge coefficients disappear.

Figure 3 plots training and validation RMS error against $\ln \lambda$. Compare it with what happened when we varied the degree instead:

> From Lecture 5
>
> In the lecture's sine experiment the training error (RMS error) falls all the way to zero at $M = 9$, while the validation error falls until about $M = 3$, stays flat to $M = 8$, and then shoots up at $M = 9$.

Figure 3 is the same picture viewed from the other side: there, flexibility increased with the degree M; here the degree is fixed at 9 and flexibility *decreases* as λ grows. Training error can only rise with λ. Validation error (which is 1.36 with no penalty, off the top of the plot) falls to a minimum of about 0.31 near $\ln \lambda = -8$ and then rises again as the model underfits.

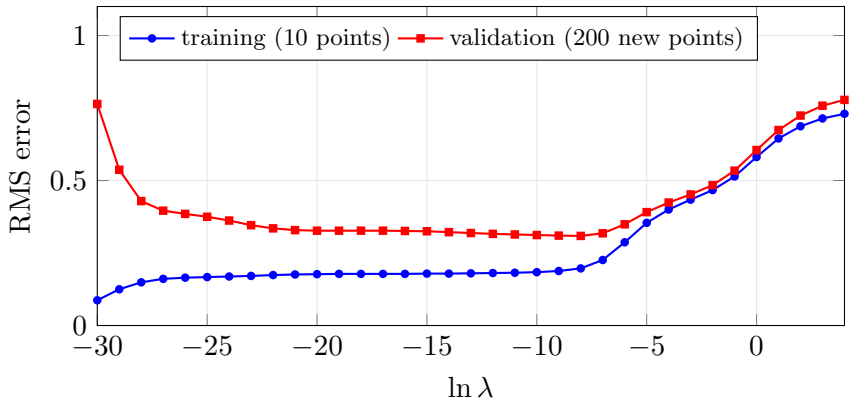


Figure 3: Training and validation RMS error of the ridge-penalized degree-9 polynomial against $\ln \lambda$ (one simulated draw). Small λ (left) overfits, large λ (right) underfits; validation error is lowest near $\ln \lambda = -8$.

Two knobs, one dial. Model flexibility can be controlled by the *structure* (the degree M, the number of features) or by the *penalty* λ. Regularization is often more convenient: λ is continuous, so we can tune flexibility smoothly instead of in integer jumps, and we can keep a rich set of features while paying only for what the data support.

5 Lasso regression

Ridge never sets a weight exactly to zero. In the age–sleep example the sleep weight shrank, crossed zero and stayed at a small positive value for every λ we tried (Figure 6, left): the model still uses sleep. Often we want more than shrinkage. When there are dozens or hundreds of candidate features, as in the battery problem of Section 8, we suspect most of them are useless, and we want the model itself to decide which ones to drop. A model that uses only a few features is also easier to interpret and to check. A small change to the penalty achieves this.

5.1 The lasso loss

The *lasso* (least absolute shrinkage and selection operator, Tibshirani, 1996) replaces the squared length by the ℓ_1 length:

$$L_\lambda(w) = \tfrac{1}{2}\|y - Xw\|_2^2 + \lambda\|w_{1:d}\|_1 = \tfrac{1}{2}\sum_{i=1}^{n} e_i^2 + \lambda\sum_{j=1}^{d}|w_j|.$$

As before the intercept is not penalized, and features should be standardized. The loss is still convex (a bowl plus a sum of V shapes, both of which only curve upwards), so it has no false minima. But $|w_j|$ has a kink at $w_j = 0$, where it has no derivative; this is why least squares uses squares rather than absolute values for the residuals. Here the kink is exactly what we want:

it is what pins weights to zero. The price is that we cannot simply set a gradient to zero, and there is no matrix formula for the general solution.

5.2 Why absolute values give exact zeros

Before any algebra, a picture explains why changing squares to absolute values produces exact zeros. Both penalties can be stated a second way: instead of charging for size, give the weights a fixed *budget*. For ridge,

$$\text{minimize } \tfrac{1}{2}\|y - Xw\|_2^2 \quad \text{subject to} \quad \|w\|_2^2 \le t.$$

For every λ there is a budget t that gives the same solution, and vice versa (larger λ corresponds to smaller t); the precise link is the method of Lagrange multipliers, which we meet with constrained optimization. The constrained form is easier to draw.

Take two standardized slopes (w_1, w_2). The data-fit term is a bowl whose contours are ellipses centred at the OLS solution. The budget $\|w\|_2^2 \le t$ is a disc. The constrained solution is the first point at which the growing ellipses touch the disc. For the lasso the budget is $|w_1| + |w_2| \le t$, a diamond with corners on the axes.

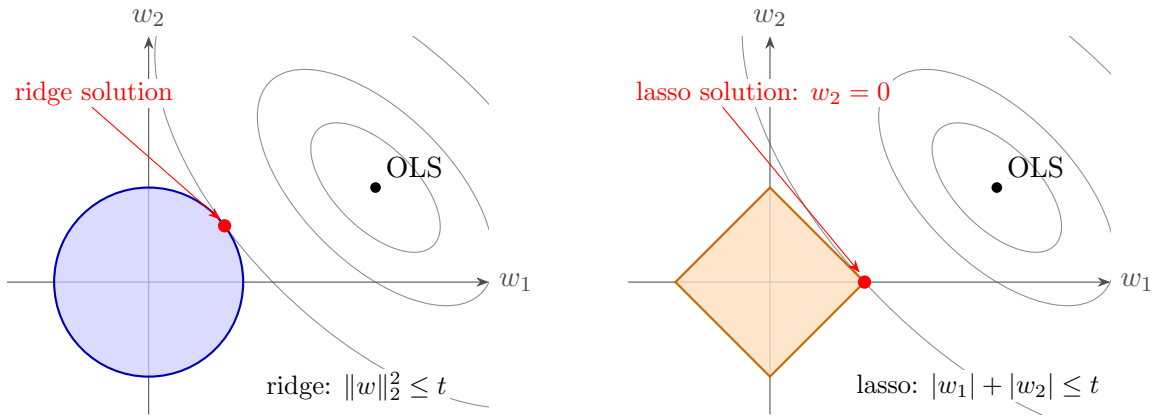


Figure 4: The same data-fit contours (grey ellipses around the OLS solution) meeting a ridge budget (left, a disc) and a lasso budget (right, a diamond) of the same size. The ridge solution lies at a generic point of the circle, with both weights non-zero. The lasso solution lies at a corner of the diamond, where $w_2 = 0$ exactly. (Illustrative quadratic, computed exactly.)

Figure 4 shows the difference. A circle is smooth: it has no preferred points, and an ellipse arriving from a generic direction touches it at a generic point, where both weights are non-zero. A diamond has *corners*, and they stick out towards the ellipses. For a wide range of ellipse positions the first contact happens at a corner, and every corner lies on an axis, where one weight is exactly zero. In d dimensions the ℓ_1 ball has corners and edges on every coordinate subspace, so lasso solutions typically have many exact zeros. In one phrase: **ridge shrinks, lasso selects.** The rest of this section confirms the picture algebraically.

5.3 One weight by hand: soft thresholding

Start with a single centred feature. Write $a = \sum_i x_i^2 > 0$ and $b = \sum_i x_i y_i$ (for our age data, $a = S_{xx} = 368.75$ and $b = S_{xy} = -37.5$). Dropping a constant, the loss is

$$f(w) = \tfrac{a}{2} w^2 - bw + \lambda |w|.$$

The function $|w|$ is smooth on each side of 0, so we treat the two sides separately and then the kink.

- **If the minimum is at some** $w > 0$, there $|w| = w$ and $f'(w) = aw - b + \lambda = 0$, so $w = (b - \lambda)/a$. This is positive only if $b > \lambda$.
- **If the minimum is at some** $w < 0$, there $|w| = -w$ and $f'(w) = aw - b - \lambda = 0$, so $w = (b + \lambda)/a$. This is negative only if $b < -\lambda$.
- **Otherwise** ($|b| \le \lambda$) the minimum is at the kink, $w = 0$. Check: just to the right of 0 the slope is $-b + \lambda \ge 0$ (uphill), and just to the left it is $-b - \lambda \le 0$ (so moving left is also uphill). Both directions go up, so 0 is the bottom.

All three cases fit in one formula. Define the *soft-thresholding* operator

$$\mathcal{S}(b, \lambda) = \operatorname{sign}(b)\, \max(|b| - \lambda,\, 0) = \begin{cases} b - \lambda & b > \lambda \\ 0 & |b| \le \lambda \\ b + \lambda & b < -\lambda. \end{cases}$$

Then

$$\boxed{w^\star_{\text{lasso}} = \frac{\mathcal{S}(b, \lambda)}{a}, \qquad \text{while} \qquad w^\star_{\text{ridge}} = \frac{b}{a + \lambda} \quad \text{and} \quad w^\star_{\text{OLS}} = \frac{b}{a}.}$$

Read the three formulas side by side. OLS divides the evidence b by a. Ridge enlarges the denominator, so it shrinks every weight by the same *factor*. Lasso subtracts λ from the evidence, so it shrinks every weight by the same *amount*, and any weight whose evidence $|b|$ is smaller than λ is set to exactly zero.

(In the language of convex analysis, the kink argument says that 0 lies in the *subdifferential* of f at $w = 0$. We do not need that vocabulary here, but you will meet it in optimization texts.)

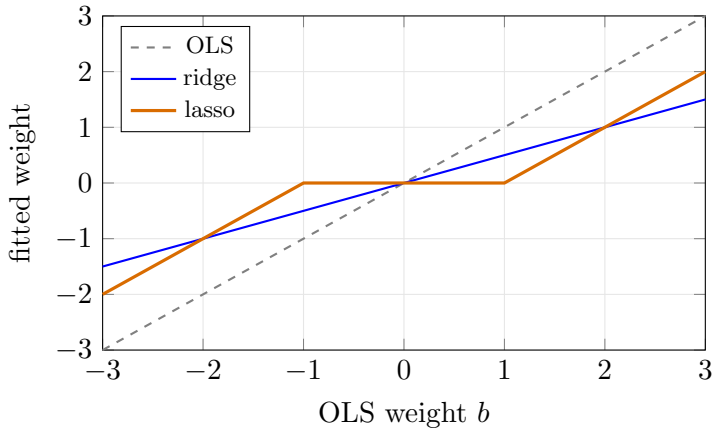


Figure 5: How each method turns the evidence b into a weight, for $a = 1$ (one standardized feature with $\sum_i x_i^2 = 1$, or orthonormal features) and $\lambda = 1$. Ridge scales every weight by $1/(1 + \lambda) = \frac{1}{2}$. Lasso subtracts $\lambda = 1$ and maps the whole interval $[-1, 1]$ to zero.

When the features are *orthonormal* ($X^\top X = I$), the problem splits into d separate one-weight problems with $a = 1$ and $b_j = x_j^\top y$, the OLS weight. Then ridge gives $b_j/(1 + \lambda)$ and lasso gives $\mathcal{S}(b_j, \lambda)$ for every j at once: Figure 5 is the complete answer.

5.4 Worked example: the age slope

For the four customers, $b = -37.5$ and $a = 368.75$, so

$$w_1(\lambda) = \frac{\mathcal{S}(-37.5, \lambda)}{368.75} = -\frac{\max(37.5 - \lambda,\, 0)}{368.75}, \qquad w_0 = 3.5 - w_1 \cdot 36.25.$$

λ	lasso slope	ridge slope (same λ)	lasso total squared error
0	-0.1017	-0.1017	1.186
10	-0.0746	-0.0990	1.458
20	-0.0475	-0.0965	2.271
30	-0.0203	-0.0940	3.627
≥ 37.5	0	(-0.0923 at 37.5)	5.000

For example, at $\lambda = 20$: $\mathcal{S}(-37.5, 20) = -17.5$ and $w_1 = -17.5/368.75 = -0.0475$. The slope falls linearly in λ and hits exactly zero at $\lambda = |b| = 37.5$, after which the lasso has decided that age is not worth using. The ridge slope at the same λ values has barely moved (its natural scale is $\lambda \sim a = 368.75$) and never reaches zero. Note that the two λ's are not on the same footing: ridge's penalty is quadratic and lasso's is linear, so "the same λ" is not "the same strength". Each method needs its own λ chosen by validation.

Check yourself. A single centred feature has $a = 2$ and $b = 5$. Find the OLS, ridge and lasso weights for $\lambda = 6$. Which λ makes the lasso weight zero?

Answer. OLS: $5/2 = 2.5$. Ridge: $5/(2 + 6) = 0.625$. Lasso: $\mathcal{S}(5, 6) = 0$ because $|5| \leq 6$, so the weight is 0. The lasso weight is zero for every $\lambda \geq 5$.

5.5 Many weights: coordinate descent

With several features the weights interact through the off-diagonal entries of $X^\mathsf{T}X$, and there is no formula. But the one-weight solution suggests an algorithm: fix all weights but one, solve for that one exactly by soft thresholding, and cycle.

Coordinate descent for the lasso (centred, standardized data). Start from $w = 0$. Repeat until the weights stop changing:
  for $j = 1, \ldots, d$:
    $r^{(j)} = y - \sum_{k \neq j} x_k w_k$ (residual ignoring feature j)
    $w_j \leftarrow \mathcal{S}\big(x_j^\mathsf{T} r^{(j)},\, \lambda\big) \big/ x_j^\mathsf{T} x_j$
Finally set $w_0 = \bar{y} - \bar{x}^\mathsf{T} w$.

Each update is the one-weight problem of Section 5.3 with $a = x_j^\mathsf{T} x_j$ and $b = x_j^\mathsf{T} r^{(j)}$, so each step can only lower the loss. Because the lasso loss is convex, the cycle converges to the global minimum. This is what most software (including scikit-learn) does. It is our first *iterative* solver, a preview of the numerical route of the next topic.

5.6 Age and sleep: the lasso chooses

Apply the lasso to the standardized age and sleep features of Section 4.1. Figure 6 (right) shows the *lasso path*. It is piecewise linear and has two events.

- From $\lambda = 0$ to $\lambda \approx 0.40$ both weights shrink linearly from the OLS values (-2.134, -1.257).
- At $\lambda \approx 0.40$ the sleep weight reaches *exactly* zero and stays there.
- The age weight then shrinks alone and reaches zero at $\lambda = 3.906$, after which the model is the constant 3.5.

Finding the knot by hand. Suppose the sleep weight is zero. Then the age weight solves a one-weight problem with $a = z_1^\mathsf{T} z_1 = 4$ and $b = z_1^\mathsf{T} y_c = -3.906$:

$$w_1 = \frac{\mathcal{S}(-3.906, \lambda)}{4} = -\frac{3.906 - \lambda}{4}.$$

Sleep stays at zero only if its own one-weight problem, given this w_1, returns zero, which by Section 5.3 means $|b_2| \le \lambda$ with

$$b_2 = z_2^{\mathsf{T}}(y_c - z_1 w_1) = 2.828 - (-3.682)\, w_1 = 2.828 - 0.9206\,(3.906 - \lambda) = -0.767 + 0.921\,\lambda.$$

For λ between 0 and 0.83 this is negative, and $|b_2| \le \lambda$ becomes $0.767 - 0.921\lambda \le \lambda$, that is $\lambda \ge 0.767/1.921 = 0.399$. So sleep is excluded exactly when $\lambda \ge 0.399$. At $\lambda = 1$, for instance, the lasso solution is $w = (-0.726,\ 0)$, and the check gives $|b_2| = |-0.767 + 0.921| = 0.153 \le 1$.

Compare the two paths in Figure 6. Faced with two strongly correlated features, ridge keeps both and spreads the weight between them; lasso picks the one with the stronger evidence (age) and discards the other. Neither is "right": with four customers we cannot know whether sleep matters. But the lasso gives a simpler model, and the decision it makes is the honest one given the data: once age is known, sleep adds almost nothing.

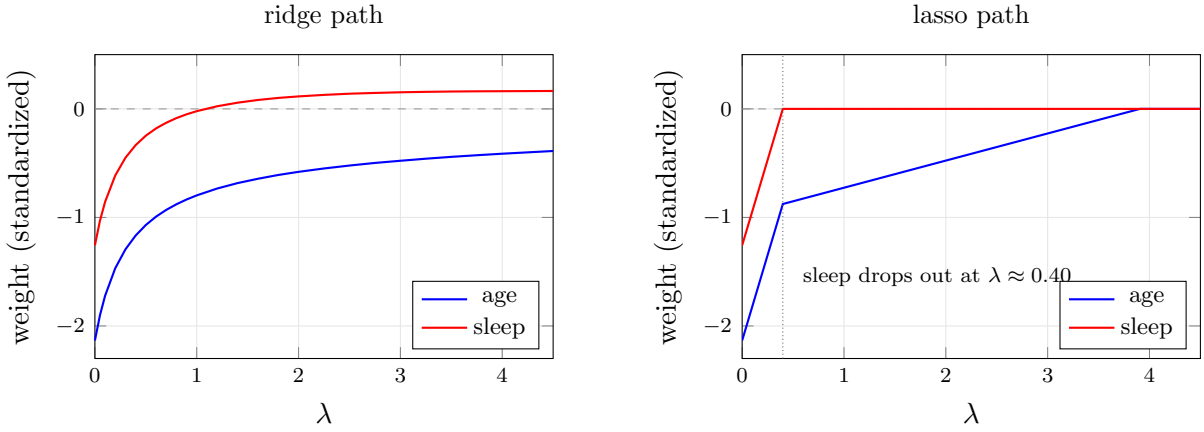


Figure 6: Weights on standardized age and sleep as λ grows. Left: ridge shrinks both smoothly; neither ever reaches zero (sleep crosses zero and changes sign). Right: lasso weights move along straight lines and hit zero one at a time.

5.7 Feature selection in a bigger example

Selection matters most when we have many candidate features and suspect that only a few matter. A simulated example makes this concrete. Each of $n = 40$ customers has $d = 12$ standardized features, but exercise truly depends on only three of them: $y = 3x_1 - 2x_4 + 1.5x_7 + \text{noise}$ (noise standard deviation 1.5). We fit OLS, ridge and lasso, choosing each λ by 5-fold cross-validation (Section 7; both picked $\lambda = 4$), and then measure RMSE on 5000 new customers (last column).

feature	1	2	3	4	5	6	7	8	9	10	11	12	RMSE
true	3.00	0	0	−2.00	0	0	1.50	0	0	0	0	0	1.49
OLS	2.92	−0.06	0.13	−2.11	−0.42	−0.10	1.38	0.16	−0.00	−0.35	0.01	0.26	1.64
ridge	2.60	−0.09	0.09	−1.87	−0.35	−0.10	1.35	0.17	−0.03	−0.18	0.10	0.27	1.65
lasso	2.77	**0**	**0**	−1.99	−0.32	**0**	1.41	0.01	**0**	−0.13	**0**	0.16	1.56

Three observations, all typical.

- **OLS** gives every one of the nine useless features a non-zero weight; with 13 parameters and 40 customers it fits a fair amount of noise.
- **Ridge** shrinks the weights but keeps all twelve. When many features each matter a little, that is the right behaviour; here, where most features are pure noise, it does not beat OLS.

- **Lasso** sets five of the nine useless weights exactly to zero, keeps all three real features, and predicts best of the three methods (the noise level, 1.5, is the best any method could achieve). It did not recover the truth perfectly: four noise features survive with small weights. Lasso selection is a useful filter, not an oracle.

6 Ridge or lasso?

We now have two penalties that behave differently on the same data. The table collects the differences seen so far, to help decide which to use.

	Ridge ($\frac{\lambda}{2}\|w\|_2^2$)	**Lasso** ($\lambda\|w\|_1$)
Solution	closed form $(X^\mathsf{T}X + \lambda I)^{-1}X^\mathsf{T}y$	iterative (coordinate descent)
Effect on weights	shrinks all by a factor; none exactly zero	shrinks by an amount; many exactly zero
Geometry	round budget (disc, ball)	budget with corners (diamond)
Uniqueness	always unique for $\lambda > 0$	can be non-unique when features are perfectly correlated
Correlated features	spreads weight across the group (equal for exact copies)	tends to pick one of the group
Best when	many features each contribute a little	a few features matter, most do not
Interpretability	all features remain	a short list of selected features

If you cannot decide, the *elastic net* uses both penalties at once, $\lambda_1\|w\|_1 + \frac{\lambda_2}{2}\|w\|_2^2$. It selects like the lasso but, thanks to the ridge part, keeps groups of correlated features together instead of picking one arbitrarily. It costs a second hyper-parameter.

7 Choosing λ

Every result so far depended on a λ that we picked by hand: $\lambda = 100$ for the age line, 0.3 for the cubic, e^{-8} for the sine wave. Figure 3 showed that the choice matters: too small and the model overfits, too large and it underfits. This section says how to choose λ from the data.

7.1 Never on training error

Training error can only increase with λ, because $\lambda = 0$ is exactly the problem of minimizing training error. Choosing λ by training error would always pick $\lambda = 0$ and undo the regularization. As with the degree M, the choice must be made on data the model did not see.

7.2 Leave-one-out on our customers

With only four customers we cannot set aside a separate validation set. Lecture 5 used *leave-one-out* (LOO) validation instead:

> From Lecture 5
>
> Four points are too few for a real validation set, so we use leave-one-out: hold out one customer, fit on the other three, and measure the error on the held-out person. Repeating for each of the four customers gives, for the root mean-squared error of the held-out predictions:
>
> | $M = 0$ (constant) | $M = 1$ (line) | $M = 2$ (parabola) |
> |---|---|---|
> | 1.49 | 1.06 | 3.57 |
>
> ($M = 3$ cannot be tried: three training points do not determine four parameters.) The line wins.

With ridge the cubic is always solvable, so we can now compare every combination of degree and penalty. Using standardized age and an unpenalized intercept, the LOO RMSE of the held-out predictions is:

	$\lambda = 0$	0.1	0.3	1	3	10
$M = 1$ (line)	1.06	1.05	**1.03**	1.04	1.15	1.33
$M = 2$ (parabola)	3.57	1.98	1.25	1.06	1.20	1.35
$M = 3$ (cubic)	—	1.77	1.60	1.38	1.29	1.32

(For simplicity the standardization uses the mean and standard deviation of all four ages; a careful analysis would recompute them inside each fold.)

Three lessons. Regularization rescues the parabola dramatically (3.57 down to 1.06) and makes the cubic usable. The best model is still the line, now with a small penalty: four customers cannot support a curve, and no penalty changes that. And the table has a *row* minimum for each degree: every model class has its own best λ, so degree and penalty are chosen together.

7.3 k-fold cross-validation in practice

LOO is expensive for large n (it fits n models). The standard compromise is *k-fold cross-validation*:

1. Split the training data into k groups (folds), typically $k = 5$ or 10.
2. For each candidate λ on a grid (usually spaced evenly in $\ln \lambda$), fit on $k - 1$ folds and measure error on the remaining one; repeat for every fold and average.
3. Choose the λ with the smallest average error, then refit on all the training data with that λ.
4. Report final performance on a separate *test* set that played no part in choosing λ.

Two cautions apply. Anything learned from data, including the means and standard deviations used for standardizing, must be computed inside the training folds only. And grouped data (several measurements of the same battery) must be split by group, not by row.

8 Regularization in practice

The formulas so far are how we derive and reason about the answers. This section covers how they are computed in practice, what they cost, and how they apply to a real problem.

8.1 Ridge as least squares with extra rows

Software never computes $(X^\mathsf{T}X)^{-1}$ explicitly. Forming $X^\mathsf{T}X$ magnifies rounding errors, so libraries instead call a least-squares routine such as NumPy's `lstsq`, which works on X directly. Ridge can use the same routine. Append d rows to the data:

$$\tilde{X} = \begin{bmatrix} X \\ 0 \quad \sqrt{\lambda}\, I_d \end{bmatrix}, \qquad \tilde{y} = \begin{bmatrix} y \\ 0 \end{bmatrix} \quad \Longrightarrow \quad \|\tilde{y} - \tilde{X}w\|_2^2 = \|y - Xw\|_2^2 + \lambda \|w_{1:d}\|_2^2.$$

(The 0 is the column of the intercept, which is not penalized.) So ridge is ordinary least squares on augmented data: d imaginary customers, each with only one feature switched on, whose "exercise" is zero. They pull every slope towards zero, harder for larger λ. For our line with $\lambda = 100$:

```
import numpy as np
age = np.array([25, 30, 40, 50.])
y   = np.array([ 4,  5,  3,  2.])

X  = np.column_stack([np.ones(4), age])
Xa = np.vstack([X, [0, np.sqrt(100.)]])      # one extra row
ya = np.append(y, 0)
w, *_ = np.linalg.lstsq(Xa, ya, rcond=None)  # [ 6.4, -0.08 ]
```

This matches the hand calculation of Section 2.4, and it also shows directly why ridge always has a unique solution: the extra rows give $\tilde{X}$ independent columns no matter what X looks like.

8.2 Cost

Ridge costs the same as OLS (form a $(d+1) \times (d+1)$ system, solve it). Lasso needs several sweeps of coordinate descent, each costing about nd operations, and in exchange the many zero weights make the final model cheaper to store and evaluate. In both cases the dominant expense in practice is cross-validation, which multiplies the cost by the number of folds times the number of λ values tried. Software reduces this with *warm starts*: the solution for one λ is the starting point for the next, and along the path neighbouring solutions are close.

8.3 The battery problem revisited

Recall the battery example:

> From Lecture 5
>
> One of the lecture's examples is predicting the remaining useful life (RUL) of a battery from its charging and discharging history. [...] Each battery gives one sample: features x such as number of charge cycles, average operating temperature, and charging rate; target y equal to the number of cycles it lasted from that point. The data are noisy because no two batteries are identical. [...] With a few dozen batteries and many added features, the training error can be driven to zero while predictions for new batteries become useless.

This is where regularization earns its keep.

1. **Many candidate features, few batteries.** Engineers can compute hundreds of features from a charging curve (voltages at fixed times, slopes, temperatures, their squares and logarithms) but may have only a few dozen batteries. Here $d > n$ is common, OLS has no unique solution, and both ridge and lasso still do.

2. **Lasso as a feature finder.** The lasso returns a short list of features that predict life, which engineers can inspect and reason about. A widely cited study of lithium-ion cells (Severson et al., *Nature Energy*, 2019) used exactly this kind of regularized linear model, an elastic net, to predict cycle life from features of the first 100 cycles.

3. **Ridge for many small effects.** If degradation depends weakly on many measurements, ridge usually predicts better than lasso.

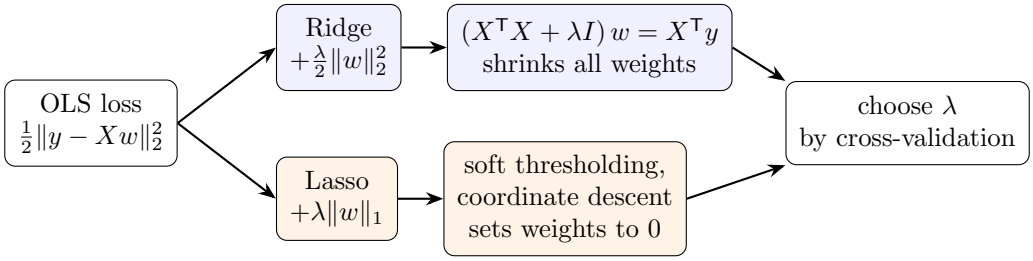
4. **Validation by battery.** Choose λ by cross-validation in which each fold holds out whole batteries.

8.4 Where this leads

Three threads continue from here.

- **Iterative optimization.** The lasso already needed an iterative solver. The next topic, gradient descent, handles any differentiable loss. For ridge, a gradient step $w \leftarrow w - \eta\big(\nabla_{\text{fit}} + \lambda w\big) = (1 - \eta\lambda)w - \eta\nabla_{\text{fit}}$ multiplies the weights by $(1 - \eta\lambda)$ before each update, which is why ridge is called *weight decay* in deep learning.
- **Probability.** Squared error is the natural loss when the noise is Gaussian. In the same way, ridge is what you get by additionally believing, before seeing data, that weights are probably small and Gaussian-distributed around zero, and lasso corresponds to a sharper, Laplace-shaped belief that many weights are exactly or nearly zero. The probability part of the course makes this precise (maximum a posteriori estimation).
- **Bias and variance.** Ridge deliberately gives slightly wrong (biased) weights in exchange for weights that change much less from one data set to the next (lower variance). The bias–variance trade-off, which formalizes the U-shaped validation curves of Figure 3, comes with the statistics of learning.

9 Summary map



Idea	Key formula or fact	Remember
Regularized loss	data fit + $\lambda\times$ size of w	$\lambda = 0$ is OLS; $\lambda \to \infty$ is the constant $\bar{y}$
Ridge	$w^\star = (X^\mathsf{T}X + \lambda D)^{-1}X^\mathsf{T}y$	Do not penalize the intercept
Always invertible	$v^\mathsf{T}(X^\mathsf{T}X + \lambda I)v \geq \lambda\|v\|^2 > 0$	Works with $d > n$ and duplicate features
Shrink factor	one feature: $w_1 = \frac{S_{xx}}{S_{xx}+\lambda}\times$ OLS slope	Features (and combinations) that barely vary shrink most
Units	penalties depend on feature scale	Standardize first, using training statistics
Geometry	budget: disc (ridge) vs diamond (lasso)	Corners on the axes give exact zeros
Lasso, one weight	$w = \mathcal{S}(b, \lambda)/a$, $\mathcal{S}(b, \lambda) = \text{sign}(b)\max(\|b\| - \lambda, 0)$	Ridge: $b/(a + \lambda)$, never zero
Lasso, many weights	coordinate descent, one soft threshold at a time	Convex, so it reaches the global minimum
Choosing λ	k-fold or leave-one-out validation	Never by training error
Practice	ridge = least squares on X with rows $\sqrt{\lambda}\, I$ appended	Solve with `lstsq`, not an explicit inverse